COMP5625M Assessment 2 - Image Caption Generation [100 marks]

The maximum marks for each part are shown in the section headers. The overall assessment carries a total of 100 marks.

This assessment is weighted 25% of the final grade for the module.

Motivation

Through this assessment, you will:

1. Understand the principles of text pre-processing and vocabulary building.
2. Gain experience working with an image-to-text model.
3. Use and compare two text similarity metrics for evaluating an image-to-text model, and understand evaluation challenges.

Setup and resources

Having a GPU will speed up the image feature extraction process. If you want to use a GPU, please refer to the module website for recommended working environments with GPUs.

Please implement the coursework using PyTorch and Python-based libraries, and refer to the notebooks and exercises provided.

This assessment will use a subset of the COCO "Common Objects in Context" dataset for image caption generation. COCO contains 330K images of 80 object categories, and at least five textual reference captions per image. Our subset consists of nearly 5070 of these images, each with five or more different descriptions of the salient entities and activities, and we will refer to it as COCO_5070.

To download the data:

1. **Images and annotations**: download the zipped file provided in the link here as `COMP5625M_data_assessment_2.zip`.

`Info only:` To understand more about the COCO dataset, you can look at the download page. We have already provided you with the "2017 Train/Val annotations (241MB)", but our image subset consists of fewer images than the original COCO dataset. **So, no need to download anything from here!**

2. **Image metadata**: as our set is a subset of the full COCO dataset, we have created a CSV file containing relevant metadata for our particular subset of images. You can also download it from Drive, "coco_subset_meta.csv", at the same link as 1.

Submission

Please submit the following:

1. Your completed Jupyter notebook file, in .ipynb format. **Do not change the file name.**
2. The .html version of your notebook; File > Download as > HTML (.html). Check that all cells have been run and all outputs (including all graphs you would like to be marked) are displayed in the .html for marking.

Final note:

> **Please include everything you would like to be marked in this notebook, including figures. Under each section, put the relevant code containing your solution. You may re-use functions you defined previously, but any new code must be in the appropriate section.** Feel free to add as many code cells as you need under each section.

Your student username (for example, `sc15jb`):

mm22afbf

Your full name:

Achmad Fauzi Bagus Firmansyah

Imports

Feel free to add to this section as needed.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
#!unzip -o -qq '/content/drive/MyDrive/Master in Leeds/Deep Learning/Assignment 2/COMP5625M_data_assessment_2.zip' -d '
```

```
import torch
import torch.nn as nn
from torchvision import transforms
import torchvision.models as models
from torch.utils.data import Dataset,DataLoader
from torch.nn.utils.rnn import pack_padded_sequence, pad_packed_sequence
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter
import os
import numpy as np
from PIL import Image
```

Detect which device (CPU/GPU) to use.

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Using device:', device)
```

```
Using device: cuda
```

The basic principle of our image-to-text model is as pictured in the diagram below, where an Encoder network encodes the input image as a feature vector by providing the outputs of the last convolutional layer of a pre-trained CNN (we use ResNet50). This pretrained network has been trained on the complete ImageNet dataset and is thus able to recognise common objects.

(Hint) You can alternatively use the COCO trained pretrained weights from PyTorch. One way to do this is use the "FasterRCNN_ResNet50_FPN_V2_Weights.COCO_V1" but use e.g., "resnet_model = model.backbone.body". Alternatively, you can use the checkpoint from your previous coursework where you finetuned to COCO dataset.

These features are then fed into a Decoder network along with the reference captions. As the image feature dimensions are large and sparse, the Decoder network includes a linear layer which downsizes them, followed by *a batch normalisation layer* to speed up training. Those resulting features, as well as the reference text captions, are then passed into a recurrent network (we will use **RNN** in this assessment).

The reference captions used to compute loss are represented as numerical vectors via an **embedding layer** whose weights are learned during training.

The Encoder-Decoder network could be coupled and trained end-to-end, without saving features to disk; however, this requires iterating through the entire image training set during training. We can make the **training more efficient by decoupling the networks**. Thus, we will:

> First extract the feature representations of the images from the Encoder

> Save these features (Part 1) such that during the training of the Decoder (Part 3), we only need to iterate over the image feature data and the reference captions.

Hint Try commenting out the feature extraction part once you have saved the embeddings. This way if you have to re-run the entire codes for some reason then you can only load these features.

Overview

> 1. Extracting image features
> 2. Text preparation of training and validation data
> 3. Training the decoder
> 4. Generating predictions on test data
> 5. Caption evaluation via BLEU score
> 6. Caption evaluation via Cosine similarity
> 7. Comparing BLEU and Cosine similarity

1 Extracting image features [11 marks]

> 1.1 Design a encoder layer with pretrained ResNet50 (4 marks)

> 1.2 Image feature extraction step (7 marks)

1.1 Design a encoder layer with pretrained ResNet50 (4 marks)

> Read through the template EncoderCNN class below and complete the class.

> You are expected to use ResNet50 pretrained on imageNet provided in the Pytorch library (torchvision.models)

```
class EncoderCNN(nn.Module):
    def __init__(self):
        """Load the pretrained ResNet-50 and replace top fc layer."""
        super(EncoderCNN, self).__init__()
        # Your code here!
        # TO COMPLETE
        # keep all layers of the pretrained net except the last layers of fully-connected ones (you are permitted to ta
        base_model=models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)
        self.features = torch.nn.Sequential(*(
            list(base_model.children())[:-1]))
        self.flatten=nn.Flatten()

    def forward(self, images):
        """Extract feature vectors from input images."""
        # TO COMPLETE
        # remember no gradients are needed

        # return features
        with torch.no_grad():
            features = self.features(images)
        features=self.flatten(features).unsqueeze(1)
        return features
```

The code above represent the Encoder part which use the features layer of ResNet-50.

The model will try to encode the image as input and give the tensor with size: 1x1x2048 as output which represent the feature layer.

```
# instantiate encoder and put into evaluation mode.
# Your code here!
encoder=EncoderCNN()
encoder.eval()
```

```
            (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
            (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
            (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
            (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
            (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
            (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
            (relu): ReLU(inplace=True)
          )
        )
        (8): AdaptiveAvgPool2d(output_size=(1, 1))
      )
      (flatten): Flatten(start_dim=1, end_dim=-1)
```

1.2 Image feature extraction step (7 marks)

Pass the images through the `Encoder` model, saving the resulting features for each image. You may like to use a `Dataset` and `DataLoader` to load the data in batches for faster processing, or you may choose to simply read in one image at a time from disk without any loaders.

Note that as this is a forward pass only, no gradients are needed. You will need to be able to match each image ID (the image name without file extension) with its features later, so we suggest either saving a dictionary of image ID: image features, or keeping a separate ordered list of image IDs.

Use this ImageNet transform provided.

```
data_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Resize(224),
    transforms.CenterCrop(224),
    transforms.Normalize((0.485, 0.456, 0.406),   # using ImageNet norms
                         (0.229, 0.224, 0.225))])
```

```
# Get unique images from the csv for extracting features - helper code
PATH_='/content/drive/MyDrive/Master in Leeds/Deep Learning/Assignment 2/'
imageList = pd.read_csv(PATH_+"coco_subset_meta.csv")

imagesUnique = sorted(imageList['file_name'].unique())
print('Number of images: ',len(imagesUnique))

df_unique_files =  pd.DataFrame.from_dict(imagesUnique)

df_unique_files.columns = ['file_name']
df_unique_files.head()
```

```
Number of images:  5068
```

	file_name
0	000000000009.jpg
1	000000000025.jpg
2	000000000030.jpg
3	000000000034.jpg
4	000000000036.jpg

```
# Define a class COCOImagesDataset(Dataset) function that takes the
# image file names and reads the image and apply transform to it
# ---> your code here! we have provided you a sketch

import os

IMAGE_DIR = "data/coco/images/"

class COCOImagesDataset(Dataset):
    def __init__(self, df, transform=None):
        self.df = df
        # --> your code here!
        self.transform=transform

    def __getitem__(self, index):
        filename = self.df.iloc[index]['file_name']
        # --> your code here!
        image=Image.open(PATH_+IMAGE_DIR+filename).convert("RGB")
        image=self.transform(image)
```

```
        return image, filename

    def __len__(self):
        return len(self.df)


# Use Dataloader to use the unique files using the class COCOImagesDataset
# make sure that shuffle is False as we are not aiming to retrain in this exercise
# Your code here-->
images_coco=COCOImagesDataset(df=df_unique_files,transform=data_transform)
images_coco_dl = DataLoader(images_coco, batch_size=1,
                        shuffle=False, num_workers=2)


# Apply encoder to extract featues and save them (e.g., you can save it using image_ids)
# Hint - make sure to save your features after running this - you can use torch.save to do this
features_map = dict()
from tqdm.notebook import tqdm
import warnings
warnings.filterwarnings("ignore")



# ---> Your code here!
encoder=encoder.to(device)
for i,data in enumerate(tqdm(images_coco_dl)):
  image,filename=data
  features_map[filename[0]]=encoder(image.to(device)).cpu()

torch.save(features_map,
    PATH_+'features2_map_'+str(i/1000)+'.pt')
```

```
100%  5068/5068 [24:23<00:00, 3.29it/s]
```

```
import glob
list_file=glob.glob(PATH_+'*.pt')


list_file
```

```
['/content/drive/MyDrive/Master in Leeds/Deep Learning/Assignment 2/vocab_2.pt',
 '/content/drive/MyDrive/Master in Leeds/Deep Learning/Assignment 2/vocab_.pt',
 '/content/drive/MyDrive/Master in Leeds/Deep Learning/Assignment 2/features_map_5.067.pt',
 '/content/drive/MyDrive/Master in Leeds/Deep Learning/Assignment 2/model_rnn_num.pt',
 '/content/drive/MyDrive/Master in Leeds/Deep Learning/Assignment 2/model_rnn_n.pt',
 '/content/drive/MyDrive/Master in Leeds/Deep Learning/Assignment 2/model_rnn_dn.pt',
 '/content/drive/MyDrive/Master in Leeds/Deep Learning/Assignment 2/model_rnn.pt',
 '/content/drive/MyDrive/Master in Leeds/Deep Learning/Assignment 2/model_rnn_d.pt',
 '/content/drive/MyDrive/Master in Leeds/Deep Learning/Assignment 2/model_lstm.pt',
 '/content/drive/MyDrive/Master in Leeds/Deep Learning/Assignment 2/model_lstm_d.pt',
 '/content/drive/MyDrive/Master in Leeds/Deep Learning/Assignment 2/model_gru.pt',
 '/content/drive/MyDrive/Master in Leeds/Deep Learning/Assignment 2/model_gru_d.pt',
 '/content/drive/MyDrive/Master in Leeds/Deep Learning/Assignment 2/model_gru2.pt',
 '/content/drive/MyDrive/Master in Leeds/Deep Learning/Assignment 2/features2_map_5.067.pt',
 '/content/drive/MyDrive/Master in Leeds/Deep Learning/Assignment 2/vocab2_.pt',
 '/content/drive/MyDrive/Master in Leeds/Deep Learning/Assignment 2/model_rnn2.pt',
 '/content/drive/MyDrive/Master in Leeds/Deep Learning/Assignment 2/model_lstm2.pt']
```

```
features_map_save_path=[i for i in list_file if  'features2_map' in i][0]


features_map=torch.load(features_map_save_path)
## try for one image
features_map['000000114854.jpg'].size()
```

```
torch.Size([1, 1, 2048])
```

2 Text preparation [23 marks]

> 2.1 Build the caption dataset (3 Marks)

> 2.2 Clean the captions (3 marks)

> 2.3 Split the data (3 marks)

> 2.4 Building the vocabulary (10 marks)

2.5 Prepare dataset using dataloader (4 marks)

2.1 Build the caption dataset (3 Marks)

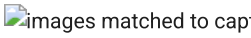
All our selected COCO_5029 images are from the official 2017 train set.

The `coco_subset_meta.csv` file includes the image filenames and unique IDs of all the images in our subset. The `id` column corresponds to each unique image ID.

The COCO dataset includes many different types of annotations: bounding boxes, keypoints, reference captions, and more. We are interested in the captioning labels. Open `captions_train2017.json` from the zip file downloaded from the COCO website. You are welcome to come up with your own way of doing it, but we recommend using the `json` package to initially inspect the data, then the `pandas` package to look at the annotations (if you read in the file as `data`, then you can access the annotations dictionary as `data['annotations']`).

Use `coco_subset_meta.csv` to cross-reference with the annotations from `captions_train2017.json` to get all the reference captions for each image in COCO_5029.

For example, you may end up with data looking like this (this is a `pandas` DataFrame, but it could also be several lists, or some other data structure/s):




```
import json

# loading captions for training
with open(PATH_+'data/coco/annotations2017/captions_train2017.json', 'r') as json_file:
    data = json.load(json_file)

df = pd.DataFrame.from_dict(data["annotations"])
df.image_id=df.image_id.astype(str)
df.head()
```

	image_id	id	caption
0	203564	37	A bicycle replica with a clock as the front wh...
1	322141	49	A room with blue walls and a white sink and door.
2	16977	89	A car that seems to be parked illegally behind...
3	106140	98	A large passenger airplane flying through the ...
4	106140	101	There is a GOL plane taking off in a partly cl...

```
# Hint: get the filename matching id from coco_subset_meta.csv - make sure that for each id you add image filename
coco_subset = pd.read_csv(PATH_+"coco_subset_meta.csv",dtype={'id':str})
# --> your code here! - name the new dataframe as "new_file"
new_file=coco_subset.merge(df[['id','image_id','caption']],
                           left_on='id',
                           right_on='image_id',
                           how='inner')[['image_id','id_y','caption','file_name']].rename(columns={'id_y':'id'})
new_file.head(3)
```

	image_id	id	caption	file_name
0	262145	694	People shopping in an open market for vegetables.	000000262145.jpg
1	262145	1054	An open market full of people and piles of veg...	000000262145.jpg
2	262145	1456	People are shopping at an open air produce mar...	000000262145.jpg

Since we notice that there is duplicate caption for an image, we drop the row with duplicate caption first

```
new_file=new_file.drop_duplicates('caption')
```

2.2 Clean the captions (3 marks)

Create a cleaned version of each caption. If using dataframes, we suggest saving the cleaned captions in a new column; otherwise, if you are storing your data in some other way, create data structures as needed.

A cleaned caption should be all lowercase, and consist of only alphabet characters.

Print out 10 original captions next to their cleaned versions to facilitate marking.

```
new_file["clean_caption"] = "" # add a new column to the dataframe for the cleaned captions
import re
def gen_clean_captions_df(df):

    # Remove spaces in the beginning and at the end
    # Convert to lower case
    # Replace all non-alphabet characters with space
    # Replace all continuous spaces with a single space

    # -->your code here
    clean_caption=df.caption.tolist()
    clean_caption=[u.strip().lower() for u in clean_caption]
    clean_caption=[re.sub('[^a-z]+', ' ', u) for u in clean_caption]
    clean_caption=[u.replace("\s"," ").lstrip().rstrip() for u in clean_caption]

    # add to dataframe

    df["clean_caption"] = clean_caption

    return df


new_file = gen_clean_captions_df(new_file)
new_file.head(10)
```

	image_id	id	caption	file_name	clean_caption
0	262145	694	People shopping in an open market for vegetables.	000000262145.jpg	people shopping in an open market for vegetables
1	262145	1054	An open market full of people and piles of veg...	000000262145.jpg	an open market full of people and piles of veg...
2	262145	1456	People are shopping at an open air produce mar...	000000262145.jpg	people are shopping at an open air produce market
3	262145	5248	Large piles of carrots and potatoes at a crowd...	000000262145.jpg	large piles of carrots and potatoes at a crowd...
4	262145	5254	People shop for vegetables like carrots and po...	000000262145.jpg	people shop for vegetables like carrots and po...
15	262146	634780	a person skiing down a steep hill	000000262146.jpg	a person skiing down a steep hill
16	262146	637393	A person skiing down a steep snowy hill.	000000262146.jpg	a person skiing down a steep snowy hill
17	262146	640348	A person on snow skis going down a steep slope.	000000262146.jpg	a person on snow skis going down a steep slope
18	262146	641836	A skier is skiing down a down hill slope.	000000262146.jpg	a skier is skiing down a down hill slope
19	262146	649789	A skier is shown taking on a very steep slope.	000000262146.jpg	a skier is shown taking on a very steep slope

However, it turns out that after cleaning the caption (lower case, replace the non alphabet character, or remove the continuous space) there is duplicateion in clean_caption. Hence we repeat the drop duplication part, but using the clean_caption column as preferences.

```
new_file[new_file.duplicated('clean_caption',keep=False)].sort_values('clean_caption').shape
```

```
(70, 5)
```

```
new_file_c=new_file.drop_duplicates('clean_caption',keep='first')
```

2.3 Split the data (3 marks)

Split the data 70/10/20% into train/validation/test sets. **Be sure that each unique image (and all corresponding captions) only appear in a single set.**

We provide the function below which, given a list of unique image IDs and a 3-split ratio, shuffles and returns a split of the image IDs.

If using a dataframe, `df['image_id'].unique()` will return the list of unique image IDs.

```
import random
import math

def split_ids(image_id_list, train=.7, valid=0.1, test=0.2):
    """
    Args:
        image_id_list (int list): list of unique image ids
        train (float): train split size (between 0 - 1)
        valid (float): valid split size (between 0 - 1)
        test (float): test split size (between 0 - 1)
    """
    list_copy = image_id_list.copy()
```

```
    random.shuffle(list_copy)

    train_size = math.floor(len(list_copy) * train)
    valid_size = math.floor(len(list_copy) * valid)
    return list_copy[:train_size], list_copy[train_size:(train_size + valid_size)], list_copy[(train_size + valid_size)
```

The code below used for subsetting the train set with specification: 70% for training, 10% for validation, and 20% for testing

```
uniqe_image_list=new_file_c.image_id.unique().tolist()
train_index, valid_index, test_index=split_ids(uniqe_image_list)
torch.save([train_index, valid_index, test_index],PATH_+'index_data2.pth')
```

```
[train_index, valid_index, test_index]=torch.load(PATH_+'index_data2.pth')
```

```
train_set=new_file_c.query('image_id in @train_index').reset_index()[new_file.columns]
valid_set=new_file_c.query('image_id in @valid_index').reset_index()[new_file.columns]
test_set=new_file_c.query('image_id in @test_index').reset_index()[new_file.columns]
```

2.4 Building the vocabulary (10 marks)

The vocabulary consists of all the possible words which can be used - both as input into the model, and as output predictions, and we will build it using the cleaned words found in the reference captions from the training set. In the vocabulary each unique word is mapped to a unique integer (a Python `dictionary` object).

A `Vocabulary` object is provided for you below to use.

```
class Vocabulary(object):
    """ Simple vocabulary wrapper which maps every unique word to an integer ID. """
    def __init__(self):
        # intially, set both the IDs and words to dictionaries with special tokens
        self.word2idx = {'<pad>': 0, '<unk>': 1, '<end>': 2}
        self.idx2word = {0: '<pad>', 1: '<unk>', 2: '<end>'}
        self.idx = 3

    def add_word(self, word):
        # if the word does not already exist in the dictionary, add it
        if not word in self.word2idx:
            # this will convert each word to index and index to word as you saw in the tutorials
            self.word2idx[word] = self.idx
            self.idx2word[self.idx] = word
            # increment the ID for the next word
            self.idx += 1

    def __call__(self, word):
        # if we try to access a word not in the dictionary, return the id for <unk>
        if not word in self.word2idx:
            return self.word2idx['<unk>']
        return self.word2idx[word]

    def __len__(self):
        return len(self.word2idx)
```

Collect all words from the cleaned captions in the **training and validation sets**, ignoring any words which appear 3 times or less; this should leave you with roughly 2200 words (plus or minus is fine). As the vocabulary size affects the embedding layer dimensions, it is better not to add the very infrequently used words to the vocabulary.

Create an instance of the `Vocabulary()` object and add all your words to it.

```
# [Hint] building a vocab function such with frequent words e.g., setting MIN_FREQUENCY = 3
def build_vocab(df_ids, new_file,MIN_FREQUENCY):
    """
    Parses training set token file captions and builds a Vocabulary object and dataframe for
    the image and caption data

    Returns:
        vocab (Vocabulary): Vocabulary object containing all words appearing more than min_frequency
    """
    word_mapping = Counter()
```

```
    # for index in df.index:
    for index, id in enumerate(df_ids):
        caption = str(new_file.loc[new_file['image_id']==id]['clean_caption'])
        for word in caption.split():
            # also get rid of numbers, symbols etc.
            if word in word_mapping:
                word_mapping[word] += 1
            else:
                word_mapping[word] = 1

    # create a vocab instance
    vocab = Vocabulary()

    # add the words to the vocabulary
    for word in word_mapping:
        # Ignore infrequent words to reduce the embedding size
        # --> Your code here!
        if word_mapping[word]>MIN_FREQUENCY:
          vocab.add_word(word)

    return vocab#,word_mapping



# build your vocabulary for train, valid and test sets
# ---> your code here!
MIN_FREQUENCY = 3
all_train_index=train_index
all_train_index.extend(valid_index)
print(f'Start building vocab with Min Frequency:{MIN_FREQUENCY}')
#while True:
vocab=build_vocab(all_train_index,new_file,MIN_FREQUENCY)
print('vocab_length: ', vocab.__len__())
#  if (vocab.__len__()/100>23):
#    MIN_FREQUENCY+=1
#    print('increase the min-frequency into:',MIN_FREQUENCY)
#  else:
#    break
print('Try Vocab "open":,', vocab.__call__('open'))
torch.save(vocab, PATH_+'vocab2_.pt')
```

```
Start building vocab with Min Frequency:3
vocab_length:  2481
Try Vocab "open":, 152
```

The code above represent the create the word which appears more than 3 times. However, the length of vocabulary is quite different with the threshold mentioned previously. The vocab length is 2481 (which is 281 greater than 2200). Yet, since we focusing in removing the frequency of the word and the differences is not so big, we will stick on this vocab.

```
vocab=torch.load(PATH_+'vocab2_.pt')
```

2.5 Prepare dataset using dataloader (4 marks)

Create a PyTorch `Dataset` class and a corresponding `DataLoader` for the inputs to the decoder. Create three sets: one each for training, validation, and test. Set `shuffle=True` for the training set DataLoader.

The `Dataset` function `__getitem__(self, index)` should return three Tensors:

1. A Tensor of image features, dimension (1, 2048).
2. A Tensor of integer word ids representing the reference caption; use your `Vocabulary` object to convert each word in the caption to a word ID. Be sure to add the word ID for the `<end>` token at the end of each caption, then fill in the the rest of the caption with the `<pad>` token so that each caption has uniform lenth (max sequence length) of **47**.
3. A Tensor of integers representing the true lengths of every caption in the batch (include the `<end>` token in the count).

Note that as each unique image has five or more (say, `n`) reference captions, each image feature will appear `n` times, once in each unique (feature, caption) pair.

```
MAX_SEQ_LEN = 47

class COCO_Features(Dataset):
    """ COCO subset custom dataset, compatible with torch.utils.data.DataLoader. """
    def __init__(self, df, features, vocab):
```

```
        """
        Args:
            df: (dataframe or some other data structure/s you may prefer to use)
            features: image features
            vocab: vocabulary wrapper
        """
        self.df=df
        self.features=features
        self.vocab=vocab
        # TO COMPLETE
    def __getitem__(self, index):
        """ Returns one data tuple (feature [1, 2048], target caption of word IDs [1, 47], and integer true caption ler
        filename=self.df.iloc[index]['file_name']
        feature_=self.features[filename]
        clean_caption=self.df.iloc[index]['clean_caption']
        target_caption=[]
        caption_=clean_caption.split(' ')
        length_=len(caption_)
        listofpads = ['<pad>'] * MAX_SEQ_LEN
        if length_>=MAX_SEQ_LEN:
          listofpads=caption_[:MAX_SEQ_LEN]
          listofpads[(MAX_SEQ_LEN-1)]='<end>'
        else:
          caption_.append('<end>')
          listofpads[:(length_+1)]=caption_[:(length_+1)]
        target_caption=[self.vocab.__call__(i) for i in listofpads]
        true_length=target_caption.index(2)+1
        return (feature_,
                torch.tensor(target_caption),
                true_length)

    def __len__(self):
        return len(self.features)


def caption_collate_fn(data):
    """ Creates mini-batch tensors from the list of tuples (image, caption).
    Args:
        data: list of tuple (image, caption).
            - image: torch tensor of shape (3, 224, 224).
            - caption: torch tensor of shape (?); variable length.
    Returns:
        images: torch tensor of shape (batch_size, 3, 224, 224).
        targets: torch tensor of shape (batch_size, padded_length).
        lengths: list; valid length for each padded caption.
    """
    data.sort(key=lambda x: x[2], reverse=True)
    images, captions,lengths_1 = zip(*data)
    # Merge images (from tuple of 3D tensor to 4D tensor).
    images = torch.stack(images)#.unsqueeze(dim=1)
    # Merge captions (from tuple of 1D tensor to 2D tensor).
    lengths = [len(cap) for cap in captions]
    targets = torch.zeros(len(captions), max(lengths)).long()
    for i, cap in enumerate(captions):
        end = lengths[i]
        targets[i, :end] = cap[:end]
    return images, targets, lengths_1
```

In the following code, we create different features map for training and validation. Since the len of dataset is affected with the length of features map, we try to give the suitable features map whether it is training or validation data.

```
train_features_map = {key: features_map[key] for key in train_set.file_name}
valid_features_map = {key: features_map[key] for key in valid_set.file_name}


dataset_train = COCO_Features(
    df=train_set,
    vocab=vocab,
    features=train_features_map,
)

#  your dataloader here (make shuffle true as you will be training RNN)
# --> your code here!
```

```
train_dl = torch.utils.data.DataLoader(dataset=dataset_train,
                                       batch_size=64,
                                       shuffle=True,
                                       num_workers= 2,
                                       collate_fn=caption_collate_fn)

# Do the same as above for your validation set
# ---> your code here!
dataset_valid = COCO_Features(
    df=valid_set,
    vocab=vocab,
    features=valid_features_map,
)

valid_dl = torch.utils.data.DataLoader(dataset=dataset_valid,
                                       batch_size=64,
                                       shuffle=True,
                                       num_workers=2,
                                       collate_fn=caption_collate_fn)
```

Load one batch of the training set and print out the shape of each returned Tensor.

```
print('Number of length in training data loader: ',train_dl.__len__())
```

```
Number of length in training data loader:  56
```

```
sample=next(iter(train_dl))
```

```
print('Shape of Image: ',sample[0][0].size())
print('Shape of Captions: ',sample[1][0].size())
print('WordIdx Captions: ',sample[1][0])
list_result=[]
for idx,i in enumerate(sample[1][0]):
  list_result.append(vocab.idx2word[i.item()])
print('Captions: ',' '.join(list_result))
print('True length of Captions: ', sample[2][0])
```

```
Shape of Image:  torch.Size([1, 1, 2048])
Shape of Captions:  torch.Size([47])
WordIdx Captions:  tensor([ 149, 1695,    8,    5,  917,   65,   22,   34,   79,    5,  797,    8,
           5,  227, 1017,   19,  365,    2,    0,    0,    0,    0,    0,    0,
           0,    0,    0,    0,    0,    0,    0,    0,    0,    0,    0,    0,
           0,    0,    0,    0,    0,    0,    0,    0,    0,    0,    0])
Captions:  an intersection with a bus motorcycle and person on a bike with a man waiting to cross <end> <pad> <pad> <pad> <pad> <pad
True length of Captions:  18
```

3 Train DecoderRNN [20 marks]

> 3.1 Design RNN-based decoder (10 marks)

> 3.2 Train your model with precomputed features (10 Marks)

3.1 Design a RNN-based decoder (10 marks)

Read through the `DecoderRNN` model below. First, complete the decoder by adding an `RNN` layer to the decoder where indicated, using the PyTorch API as reference.

Keep all the default parameters except for `batch_first`, which you may set to True.

In particular, understand the meaning of `pack_padded_sequence()` as used in `forward()`. Refer to the PyTorch `pack_padded_sequence()` documentation.

Decoder

```
class DecoderNet(nn.Module):

    def __init__(self, vocab_size,num_features = 2048, embed_size=256, hidden_size=512, num_layers=1, max_seq_length=47
        """Set the hyper-parameters and build the layers."""
        super(DecoderNet, self).__init__()
        self.num_features=num_features
        self.hidden_size=hidden_size
```

```
        self.embed_size=embed_size

        # we want a specific output size, which is the size of our embedding, so
        # we feed our extracted features from the last convolutional layer (flattened to dimensions after AdaptiveAvgPo
        # other layers are also accepted but this will affect your accuracy!)
        # into a Linear layer to resize
        # your code
        self.resize=nn.Sequential(
            nn.Flatten(),
            nn.Linear(num_features,256),
            )
        #self.drop=nn.Droupout(0.2)
        # batch normalisation helps to speed up training
        # your code
        self.bn=nn.BatchNorm1d(embed_size,momentum=0.01)
        # your code for embedding layer
        self.embed = nn.Sequential(
            nn.Embedding(vocab_size, embed_size))#,
            #nn.Dropout(0.1))
        # your code for RNN
        if type_decoder=='lstm':
          self.decode = nn.LSTM(embed_size, hidden_size, num_layers,batch_first=True)
        if type_decoder=='gru':
          self.decode = nn.GRU(embed_size, hidden_size, num_layers,batch_first=True)
        else:
          self.decode = nn.RNN(embed_size, hidden_size, num_layers,batch_first=True)
        # self.linear: linear layer with input: hidden layer, output: vocab size
        # --> your code
        self.linear = nn.Sequential(
            nn.Linear(hidden_size, vocab_size))
        self.max_seq_length = max_seq_length

    def forward(self, features, captions, lengths):
      # Initialize the hidden state

        """Decode image feature vectors and generates captions."""
        # Discard the <end> word to avoid predicting when <end> is the input of the RNN
        embeddings = self.embed(captions)
        # compute your feature embeddings
        # your code
        im_features = self.resize(features)
        im_features = self.bn(im_features)

        embeddings = torch.cat([im_features.unsqueeze(1), embeddings], 1)
        # pack_padded_sequence returns a PackedSequence object, which contains two items:
        # the packed data (data cut off at its true length and flattened into one list), and
        # the batch_sizes, or the number of elements at each sequence step in the batch.
        # For instance, given data [a, b, c] and [x] the PackedSequence would contain data
        # [a, x, b, c] with batch_sizes=[2,1,1].

        # your code [hint: use pack_padded_sequence]
        packed = pack_padded_sequence(embeddings, lengths, batch_first=True,enforce_sorted=False)
        decode_out, _ = self.decode(packed)
        outputs = self.linear(decode_out[0])
        return outputs

    def sample(self, features, states=None):
        """Generate captions for given image features using greedy search."""
        sampled_ids = []
        inputs = self.bn(self.resize(features)).unsqueeze(1)
        #inputs = features.unsqueeze(1)

        for i in range(self.max_seq_length):
            hiddens, states = self.decode(inputs, states)  # hiddens: (batch_size, 1, hidden_size)
            outputs = self.linear(hiddens.squeeze(1))   # outputs:  (batch_size, vocab_size)
            _, predicted = outputs.max(1)               # predicted: (batch_size)
            sampled_ids.append(predicted)
            inputs = self.embed(predicted)              # inputs: (batch_size, embed_size)
            inputs = inputs.unsqueeze(1)                # inputs: (batch_size, 1, embed_size)

        sampled_ids = torch.stack(sampled_ids, 1)       # sampled_ids: (batch_size, max_seq_length)
        return sampled_ids
# instantiate decoder
```

```
# your code here!
```

The model above is the decoder used. The specification are below:

1. It consist the resize layer for change the size of features layer, which previously 2048 matching the embed size 256. Inside of it there is Flatten layer (for flattening the layer from 3-D tensor into 2-D tensor) and linear layer (for resizing the layer from 2048 into 256).
2. Next, there is bathcNorm1d which used for increasing the training speed by performing the standardization of the resized features layer.
3. Embedding layer which used for creating the embedding vector representing the mapping of caption into vector spaces.
4. The RNN/LSTM layer which used for training the sequence of embedding and produce the hidden layer which have size equals to 512.
5. Lastly linear layer, for training the hidden layer and resize it to match the min_sequences (47)

As written in the fourth point; we use either RNN and LSTM depends on the input. Here we try to check whether RNN and LSTM performs well in this data.

Initiate the model

```
decoderRnn=DecoderNet(vocab.__len__())
decoderLstm=DecoderNet(vocab.__len__(),type_decoder='lstm')
```

3.2 Train your model with precomputed features (10 marks)

Train the decoder by passing the features, reference captions, and targets to the decoder, then computing loss based on the outputs and the targets. Note that when passing the targets and model outputs to the loss function, the targets will also need to be formatted using `pack_padded_sequence()`.

We recommend a batch size of around 64 (though feel free to adjust as necessary for your hardware).

We strongly recommend saving a checkpoint of your trained model after training so you don't need to re-train multiple times.

Display a graph of training and validation loss over epochs to justify your stopping point.

```
nepochs=50
loss_fn=nn.CrossEntropyLoss().cuda()
```

```
import time
def training_netlayer(net,device, nepochs, results_path, train_loader, valid_loader, learning_rate,is_resume,shrink_fac
  net=net.to(device)

  stats_rec=np.zeros((2,nepochs))
  optimizer=torch.optim.Adam(net.parameters(),lr=learning_rate,weight_decay=1e-5)
  start=0
  endtime=None
  checker=0
  patience=0
  optimizer_state=None
  model_state=None
  epoc_saved=0
  best=100
  if is_resume:
    checkpoint=torch.load(results_path)
    start=checkpoint['epoch']+1
    net.load_state_dict(checkpoint['model_state_dict'])
    optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
    for i in range(start):
      stats_rec[:,i]=checkpoint['stats_rec'][:,i]
    (ltrn, lvld) =stats_rec[:,(start-1)]
    print(f"Resuming Training process- epoch: {start} training loss: {ltrn: .3f} valid loss: {lvld: .3f}")
    if 'starttime' in checkpoint.keys():
        starttime=checkpoint['starttime']
    best=lvld[start-1]

  for epoch in range(start,nepochs):
    starttime=time.time()
    running_loss = 0   # accumulated loss (for mean loss)
    n = 0              # number of minibatches
    net.train()
    for data in train_loader:
        images, caption,lengths = data
        # Zero the parameter gradients
```

```
            # Assign input and label to GPU/CPU
            images=images.to(device)
            caption=caption.to(device)
            #lengths=torch.tensor(lengths).to(device)
            # Forward, backward, and update parameters
            outputs = net(images, caption,lengths)
            target=pack_padded_sequence(caption,lengths,batch_first=True,enforce_sorted=False)
            loss= loss_fn(outputs,target[0].long())

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            # accumulate loss
            loss.cpu()
            running_loss += loss.item()
            n += 1
            #print(running_loss)
        ltrn=running_loss/n
        lvld=eval(net,valid_dl,optimizer)
        stats_rec[:,epoch] = (ltrn, lvld )
        print(f'Train {epoch+1}/{nepochs} finished. Training loss={ltrn}; Validation loss={lvld}.',end=' ')
        model_state=net.state_dict()
        optimizer_state=optimizer.state_dict()
        epoc_saved=epoch
        torch.save({"model_state_dict": model_state,
                    "epoch":epoch,
                    "optimizer_state_dict": optimizer_state,
                    "stats_rec": stats_rec,
                    'starttime':starttime,
                    'éndtime':endtime},
                   results_path)

        if best<lvld:
          print('The validation Loss does not improve.',end=' ')
          patience+=1
          if patience <=10:
            for g in optimizer.param_groups:
              g['lr'] =g['lr']*shrink_factor
              learning_rate=g['lr']
              print(f'{patience}-th Try: Shrink the learning rate into: {learning_rate}')
          else:
            print('Since no improvement made after several shrinking, terminating the process')
            break
        else:
          print('The validation Loss has been improved.')
          best=lvld
      print(f'Process is Terminated. The model and optimizer state saved based on the {epoc_saved+1}-th epoch')
      return stats_rec


    def eval(net,loader,optimizer):
      net.eval()
      running_loss=0
      with torch.no_grad():
        n=0
        for data in valid_dl:
          images, caption,lengths = data
          # Assign input and label to GPU/CPU
          images=images.to(device)
          caption=caption.to(device)
          #lengths=torch.tensor(lengths).to(device)
          outputs = net(images, caption,lengths)
          target=pack_padded_sequence(caption,lengths,batch_first=True,enforce_sorted=False)
          running_loss+=loss_fn(outputs,target[0].long()).cpu()
          n+=1
      return running_loss/n
```

The code block above represent the training and evaluation setting. For the training part, there is one specification made:

- if the validation loss is not improved from the previous epoch we try to shrink the learning rate with factor of 0.1
- the maximum number of shrinking is 10, hence if it pass 10, the training will be terminated. We choose this because the validation loss does not improved.

```
result_path=PATH_+'model_rnn2.pt'
decRNN=training_netlayer(decoderRnn,device, nepochs, result_path, train_dl, valid_dl, 0.001,False)
```

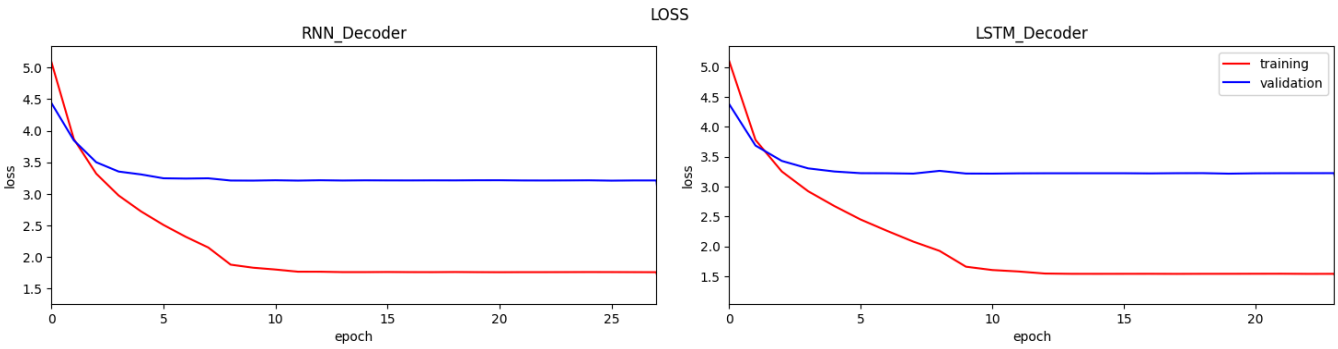
```
Train 1/50 finished. Training loss=5.08917453459331; Validation loss=4.438713550567627. The validation Loss has been improved.
Train 2/50 finished. Training loss=3.8677503977503096; Validation loss=3.848079204559326. The validation Loss has been improved.
Train 3/50 finished. Training loss=3.3198663762637546; Validation loss=3.500882387161255. The validation Loss has been improved.
Train 4/50 finished. Training loss=2.9737929020609175; Validation loss=3.3535268306732178. The validation Loss has been improved.
Train 5/50 finished. Training loss=2.721836085830416; Validation loss=3.307823419570923. The validation Loss has been improved.
Train 6/50 finished. Training loss=2.507593095302582; Validation loss=3.2468161582946777. The validation Loss has been improved.
Train 7/50 finished. Training loss=2.318604435239519; Validation loss=3.242244243621826. The validation Loss has been improved.
Train 8/50 finished. Training loss=2.14975083725793; Validation loss=3.246521472930908. The validation Loss does not improve. 1-th T
Train 9/50 finished. Training loss=1.8779048728091376; Validation loss=3.2112433910369873. The validation Loss has been improved.
Train 10/50 finished. Training loss=1.8295585193804331; Validation loss=3.210033893585205. The validation Loss has been improved.
Train 11/50 finished. Training loss=1.8007754960230418; Validation loss=3.215407133102417. The validation Loss does not improve. 2-t
Train 12/50 finished. Training loss=1.766449130007199; Validation loss=3.21018648147583. The validation Loss has been improved.
Train 13/50 finished. Training loss=1.7659411025898797; Validation loss=3.2156176567077637. The validation Loss does not improve. 3
Train 14/50 finished. Training loss=1.7599245948450906; Validation loss=3.211519718170166. The validation Loss has been improved.
Train 15/50 finished. Training loss=1.7599718017237527; Validation loss=3.2143592834472656. The validation Loss does not improve. 4
Train 16/50 finished. Training loss=1.7615132353135519; Validation loss=3.213205099105835. The validation Loss has been improved.
Train 17/50 finished. Training loss=1.7597801919494356; Validation loss=3.212843418121338. The validation Loss has been improved.
Train 18/50 finished. Training loss=1.759367429784366; Validation loss=3.2140002250671387. The validation Loss does not improve. 5-t
Train 19/50 finished. Training loss=1.7612551216568266; Validation loss=3.2134244441986084. The validation Loss has been improved.
Train 20/50 finished. Training loss=1.759362090911184; Validation loss=3.215409994125366. The validation Loss does not improve. 6-th
Train 21/50 finished. Training loss=1.7582965408052718; Validation loss=3.2157142162323. The validation Loss does not improve. 7-th
Train 22/50 finished. Training loss=1.7593049960477012; Validation loss=3.2124860286712646. The validation Loss has been improved.
Train 23/50 finished. Training loss=1.7593666187354497; Validation loss=3.2119107246398926. The validation Loss has been improved.
Train 24/50 finished. Training loss=1.759978554078511; Validation loss=3.213000535964966. The validation Loss does not improve. 8-th
Train 25/50 finished. Training loss=1.760751771075385; Validation loss=3.214625358581543. The validation Loss does not improve. 9-th
Train 26/50 finished. Training loss=1.7602778992482595; Validation loss=3.209512710571289. The validation Loss has been improved.
Train 27/50 finished. Training loss=1.7594096128429686; Validation loss=3.2125935554504395. The validation Loss does not improve. 10
Train 28/50 finished. Training loss=1.7584699200732368; Validation loss=3.212614059448242. The validation Loss does not improve. Sin
Process is Terminated. The model and optimizer state saved based on the 28-th epoch
```

```
result_path=PATH_+'model_lstm2.pt'
decLstm=training_netlayer(decoderLstm,device, nepochs, result_path, train_dl, valid_dl, 0.001,False)
```

```
Train 1/50 finished. Training loss=5.096355327538082; Validation loss=4.378077507019043. The validation Loss has been improved.
Train 2/50 finished. Training loss=3.7798800425870076; Validation loss=3.686323642730713. The validation Loss has been improved.
Train 3/50 finished. Training loss=3.2530021710055217; Validation loss=3.4295334815979004. The validation Loss has been improved.
Train 4/50 finished. Training loss=2.9232644821916307; Validation loss=3.3051137924194336. The validation Loss has been improved.
Train 5/50 finished. Training loss=2.6748367249965668; Validation loss=3.2529046535491943. The validation Loss has been improved.
Train 6/50 finished. Training loss=2.449317557471139; Validation loss=3.224836587905884. The validation Loss has been improved.
Train 7/50 finished. Training loss=2.2610381713935306; Validation loss=3.2235825061798096. The validation Loss has been improved.
Train 8/50 finished. Training loss=2.08059447152274; Validation loss=3.2176575660705566. The validation Loss has been improved.
Train 9/50 finished. Training loss=1.9254110498087746; Validation loss=3.2635629177093506. The validation Loss does not improve. 1-t
Train 10/50 finished. Training loss=1.661501094698906; Validation loss=3.2187106609344482. The validation Loss has been improved.
Train 11/50 finished. Training loss=1.6058080622128077; Validation loss=3.2175698280334473. The validation Loss has been improved.
Train 12/50 finished. Training loss=1.5816849427563804; Validation loss=3.222094774246216. The validation Loss does not improve. 2-t
Train 13/50 finished. Training loss=1.5472410661833627; Validation loss=3.223261833190918. The validation Loss does not improve. 3-t
Train 14/50 finished. Training loss=1.542498795049531; Validation loss=3.223491668701172. The validation Loss does not improve. 4-th
Train 15/50 finished. Training loss=1.5420988372394018; Validation loss=3.2237279415130615. The validation Loss does not improve. 5
Train 16/50 finished. Training loss=1.5424836533410209; Validation loss=3.2237067222595215. The validation Loss has been improved.
Train 17/50 finished. Training loss=1.542850524187088; Validation loss=3.2211062908172607. The validation Loss has been improved.
Train 18/50 finished. Training loss=1.5415505532707487; Validation loss=3.2243893146514893. The validation Loss does not improve. 6
Train 19/50 finished. Training loss=1.5425768601042884; Validation loss=3.225289821624756. The validation Loss does not improve. 7-t
Train 20/50 finished. Training loss=1.5427462799208504; Validation loss=3.216918706893921. The validation Loss has been improved.
Train 21/50 finished. Training loss=1.5434812158346176; Validation loss=3.222633123397827. The validation Loss does not improve. 8-t
Train 22/50 finished. Training loss=1.5439508195434297; Validation loss=3.2243027687072754. The validation Loss does not improve. 9
Train 23/50 finished. Training loss=1.5415971470730645; Validation loss=3.224656343460083. The validation Loss does not improve. 10
Train 24/50 finished. Training loss=1.542255889092173; Validation loss=3.225329637527466. The validation Loss does not improve. Sinc
Process is Terminated. The model and optimizer state saved based on the 24-th epoch
```

```
def graph_result(path_,title_,ax):
  results_path = path_
  data = torch.load(results_path)
  statsrec = data["stats_rec"][:,0:]
  cuttingup_=-1
  cuttingy=-1
  check=np.where(statsrec[0]==0)
  if len(check[0])>0:
    cuttingup_=check[0][0]-1
    cuttingy=min(statsrec[0][cuttingup_],statsrec[1][cuttingup_])-0.5
    if cuttingy<0:
      cuttingy=0
  ax.plot(statsrec[0], 'r', label = 'training', )
  ax.plot(statsrec[1], 'b', label = 'validation' )
  ax.set_xlabel('epoch')
  ax.set_ylabel('loss')
  ax.set_title(title_)
  if cuttingup_>-1:
    ax.set_xlim(left=0,right=cuttingup_)
```

```
    ax.set_ylim(bottom=cuttingy)
  else:
    ax.set_xlim(left=0)


fig,ax=plt.subplots(1,2,figsize=(15,4))
graph_result(PATH_+'model_rnn2.pt','RNN_Decoder',ax[0])
graph_result(PATH_+'model_lstm2.pt','LSTM_Decoder',ax[1])
ax[1].legend()
fig.tight_layout(pad=2)
fig.suptitle('LOSS')
fig.show()
```



The graph above representing the loss of training and validation between RNN and LSTM. For both training it looks that the loss drastically decreased until around 9 epoch and after 10 epoch the loss is not decreased anymore.

If we check the stagnation of loss in after 10 epoch, it seems that both decoder try to shrink its learning rate in training phase. However it turns out that the improvement is quite low hence after 25 epoch the process is terminated.

Comparing the RNN and LSTM, the validation loss of RNN is lower than LSTM yet the training loss of LSTM is lower. The difference of training loss of LSTM toward RNN is noticeable while for the validation loss is too low.

```
checkpoint=torch.load(PATH_+'model_rnn2.pt')
decoderRnn.load_state_dict(checkpoint['model_state_dict'])
decoderRnn.to(device)
decoderRnn.eval()
checkpoint=torch.load(PATH_+'model_lstm2.pt')
decoderLstm.load_state_dict(checkpoint['model_state_dict'])
decoderLstm.to(device)
decoderLstm.eval()
```

```
DecoderNet(
  (resize): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=2048, out_features=256, bias=True)
  )
  (bn): BatchNorm1d(256, eps=1e-05, momentum=0.01, affine=True, track_running_stats=True)
  (embed): Sequential(
    (0): Embedding(2481, 256)
  )
  (decode): RNN(256, 512, batch_first=True)
  (linear): Sequential(
    (0): Linear(in_features=512, out_features=2481, bias=True)
  )
)
```

4 Generate predictions on test data [8 marks]

Display 5 sample test images containing different objects, along with your model's generated captions and all the reference captions for each.

> Remember that everything **displayed** in the submitted notebook and .html file will be marked, so be sure to run all relevant cells.

```
test_features_map = {key: features_map[key] for key in test_set.file_name}
test_=list(test_features_map.keys())
```

```
test_set
```

	image_id	id	caption	file_name	clean_caption
0	262148	284571	The skateboarder is putting on a show using th...	000000262148.jpg	the skateboarder is putting on a show using th...
1	262148	286347	A skateboarder pulling tricks on top of a picn...	000000262148.jpg	a skateboarder pulling tricks on top of a picn...
2	262148	286899	A man riding on a skateboard on top of a table.	000000262148.jpg	a man riding on a skateboard on top of a table
3	262148	287571	A skate boarder doing a trick on a picnic table.	000000262148.jpg	a skate boarder doing a trick on a picnic table
4	262148	288021	A person is riding a skateboard on a picnic ta...	000000262148.jpg	a person is riding a skateboard on a picnic ta...
...	...	...	...	...	...
5029	522425	386712	A person in front of a mirror with a toothbrush.	000000522425.jpg	a person in front of a mirror with a toothbrush
5030	522425	390753	A woman is brushing her teeth as she looks in ...	000000522425.jpg	a woman is brushing her teeth as she looks in ...
5031	522425	393060	a woman with a tooth brush in her mouth stares...	000000522425.jpg	a woman with a tooth brush in her mouth stares...
5032	522425	394257	A woman brushing her teeth while looking in th...	000000522425.jpg	a woman brushing her teeth while looking in th...
5033	522425	395646	a girl looking in the mirror brushing her teeth	000000522425.jpg	a girl looking in the mirror brushing her teeth

5034 rows × 5 columns

```
test_features_map[test_set.iloc[0]['file_name']]
```

```
tensor([[[0.2193, 0.0000, 0.0000,  ..., 0.1518, 0.0000, 0.9833]]])
```

```
def generate_prediction(filename,features_map,net):
  sample_=net.sample(features_map[filename].cpu())
  list_result=[]
  for idx,i in enumerate(sample_[0]):
    if i.item()!=2:
      list_result.append(vocab.idx2word[i.item()])
  return ' '.join(list_result)
```

Sample prediction

```
from PIL import Image
image_name=test_set.iloc[500]['file_name']
filename_=PATH_+"data/coco/images/"+image_name
image = Image.open(filename_)
image.show()
print('RNN',generate_prediction(image_name,test_features_map,decoderRnn))
print('LSTM',generate_prediction(image_name,test_features_map,decoderLstm))
print('Cleaned Caption:',test_set.query('file_name==@image_name')['clean_caption'].to_list())
```


```
RNN a passenger train is on a train track
LSTM a yellow and blue train is on a track
Cleaned Caption: ['a man that is walking next to a train', 'a man walking next to a blue and yellow train', 'a man wearing a backpac
```

From the image above, it looks that RNN and LSTM can give quite good caption prediction.

For RNN: it looks it can predict the passenger

For LSTM: inspite of predicting the passenger, it give the detail regarding color of the trian.

Compared with the caption; it looks like, there is one misscaptioning. The references say "the person wearing backpack is walking besides of the **bus**". Instead of bus, both model can predict the true value which is train.

```
list_unique_image=test_set.file_name.unique().tolist()
dict_sample={}
for i in range(5):
  file_name=list_unique_image[i]
  dict_sample[i]={}
  dict_sample[i]['file_name']=PATH_+"data/coco/images/"+file_name
  dict_sample[i]['rnn_pred']=generate_prediction(file_name,features_map,decoderRnn)
  dict_sample[i]['lstm_pred']=generate_prediction(file_name,features_map,decoderLstm)
  #dict_sample[i]['gru_pred']=generate_prediction(file_name,features_map,decoderGru)
  image=file_name
  dict_sample[i]['clean_sample']=test_set.query('file_name==@image')['clean_caption'].to_list()
```

```
from PIL import Image
from io import BytesIO
from IPython.display import HTML
import base64
def get_thumbnail(path):
    i = Image.open(path)
    i.thumbnail((150, 150), Image.LANCZOS)
    return i
def image_base64(im):
    if isinstance(im, str):
        im = get_thumbnail(im)
    with BytesIO() as buffer:
        im.save(buffer, 'jpeg')
        return base64.b64encode(buffer.getvalue()).decode()

def image_formatter(im):
    return f'<img src="{image_base64(im)}">'
y_=pd.DataFrame(dict_sample).T.sample(5,random_state=1234)
y_['rnn_pred']=y_.rnn_pred.str.replace('<','&lt').replace('>','&gt')
y_['lstm_pred']=y_.lstm_pred.str.replace('<','&lt').replace('>','&gt')
#y_['gru_pred']=y_.gru_pred.str.replace('<','&lt').replace('>','&gt')
HTML(y_.to_html(formatters={'file_name': image_formatter}, escape=False))
```

	file_name	rnn_pred	lstm_pred	clean_sample
670		a man standing in front of a large body of water water	a man in a blue shirt and a woman on a motorcycle	[a woman holding a baby in a kitchen, the woman stands in a kitchen holding a child, a woman holds a child close to herself in a sun filled kitchen, the woman is holding the baby in the kitchen, a women who is holding a young child]
498		a stop sign is shown on the side of a road	a clock tower in front of a <unk> building	[a man flying through the air above a pile of snow, a man on a snowboard jumping over a teepee while people watch, a man is skiing on a steep snow covered ramp, a person high up in the air while snowboarding, a white tee pee is up and people are standing around in front of it]
322		two people are walking together on a city street	two <unk> on a grassy area in a <unk>	[two wooly sheep are on top of a hill, two fluffy blue sheep grazing in a meadow, two sheep are standing together grazing in a field, two furry animals sitting in some pine straw, two wooly animals are out in the field]
945		a <unk> of <unk> are shown in the snow	a <unk> <unk> a <unk> <unk> of a <unk> <unk>	[a table topped with cup up q tips sitting next to a pair of scissors, a pair of scissors being used to cut cotton swabs, a blanket with skinny straws and s pair of scissors on it, scissors sitting next to white sticks and a blanket, a pair of scissors sitting on some white black yellow and red material]
474		a group of people on a boat in the water	a couple of people standing around a table	[a group of people sit on top of a grassy hill, a group of people are in a green field, some people watch as a frisbee is being tossed by a man, a group of people sitting on a grass field while a man throws a frisbee, two men throwing a frizbe in front of four men]

The table above represent the comparison between RNN and LSTM in five images, randomly drawn from the test data frame.

When look at their prediction, the prediction of RNN is quite better than LSTM.

First, it can predict the "man play skateboard" and "people who riding motorcycle" better than the result of LSTM

5 Caption evaluation using BLEU score [10 marks]

There are different methods for measuring the performance of image to text models. We will evaluate our model by measuring the text similarity between the generated caption and the reference captions, using two commonly used methods. Ther first method is known as *Bilingual Evaluation Understudy (BLEU)*.

> 5.1 Average BLEU score on all data (5 marks)

> 5.2 Examplaire high and low score BLEU score samples (5 marks, at least two)

5.1 Average BLEU score on all data (5 marks)

One common way of comparing a generated text to a reference text is using BLEU. This article gives a good intuition to how the BLEU score is computed: https://machinelearningmastery.com/calculate-bleu-score-for-text-python/, and you may find an implementation online to use. One option is the NLTK implementation `nltk.translate.bleu_score` here: https://www.nltk.org/api/nltk.translate.bleu_score.html

> **Tip:** BLEU scores can be weighted by ith-gram. Check that your scores make sense; and feel free to use a weighting that best matches the data. We will not be looking for specific score ranges; rather we will check that the scores are reasonable and meaningful given the captions.

Write the code to evaluate the trained model on the complete test set and calculate the BLEU score using the predictions, compared against all five references captions.

Display a histogram of the distribution of scores over the test set.

```
from tqdm import tqdm
list_unique_image=test_set.file_name.unique().tolist()
dict_sample={}
decoderRnn.cpu()
decoderLstm.cpu()
for i in tqdm(range(len(list_unique_image))):
```

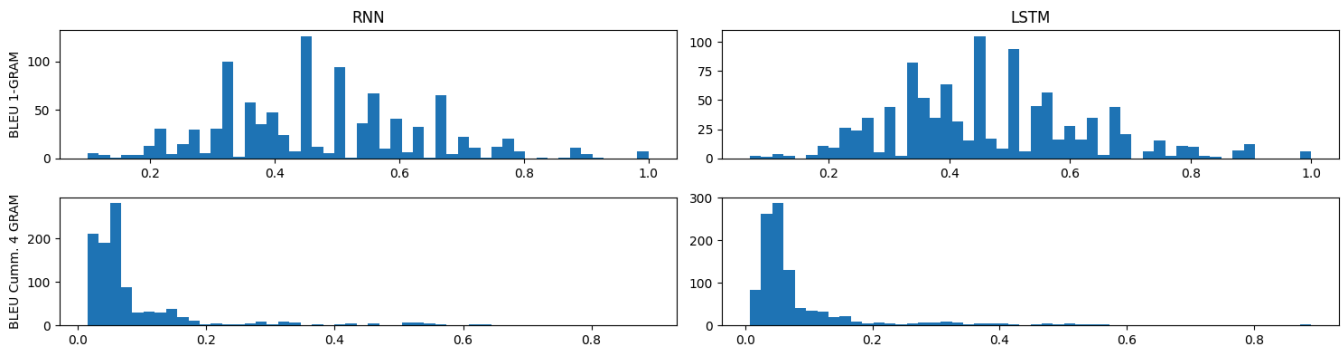
```
  file_name=list_unique_image[i]
  dict_sample[i]={}
  dict_sample[i]['file_name']=PATH_+"data/coco/images/"+file_name
  dict_sample[i]['rnn_pred']=generate_prediction(file_name,features_map,decoderRnn)
  dict_sample[i]['lstm_pred']=generate_prediction(file_name,features_map,decoderLstm)
  image=file_name
  dict_sample[i]['clean_sample']=test_set.query('file_name==@image')['clean_caption'].to_list()
```

```
100%|██████████| 1015/1015 [00:50<00:00, 19.93it/s]
```

```
test_predict_=pd.DataFrame(dict_sample).T
test_predict_.head()
```

	file_name	rnn_pred	lstm_pred	clean_sample
0	/content/drive/MyDrive/Master in Leeds/Deep Le...	a man riding a skateboard on a beach	a man in a black outfit sitting on a bench in ...	[the skateboarder is putting on a show using t...
1	/content/drive/MyDrive/Master in Leeds/Deep Le...	a man sitting on a bench in the water water	a man with a <unk> rock in a <unk>	[a man stands with a frown on his face, a old ...
2	/content/drive/MyDrive/Master in Leeds/Deep Le...	a man in a black shirt and black shirt holding...	a man in a black shirt rides a motorcycle with...	[a young man riding a skateboard on top of a p...

```
from nltk.translate.bleu_score import SmoothingFunction,sentence_bleu
chencherry = SmoothingFunction()

def bleu_calc(df,col_ref,col_hyp):
  temp=pd.DataFrame(columns=['ref','preds','bleu_1','bleu_4','cos_sim','file_name'])
  temp['file_name']=df['file_name']
  temp['ref']=df[col_ref].apply(lambda y: [i.split(' ') for i in y])
  temp['preds']=df[col_hyp].apply(lambda y: y.split(' '))
  temp['bleu_1']=temp.apply(lambda y:sentence_bleu(y['ref'],y['preds'],weights=[1],smoothing_function=chencherry.metho
  temp['bleu_4']=temp.apply(lambda y:sentence_bleu(y['ref'],y['preds'],smoothing_function=chencherry.method1),axis=1)
  temp['unk']=temp.preds.apply(lambda y: len([i for i in y if i=="<unk>"]))
  return temp


stats_rnn=bleu_calc(test_predict_,'clean_sample','rnn_pred')
stats_lstm=bleu_calc(test_predict_,'clean_sample','lstm_pred')


fig,ax=plt.subplots(2,2,figsize=(15,4))
ax[0][0].hist(stats_rnn['bleu_1'],bins=50)
ax[0][1].hist(stats_lstm['bleu_1'],bins=50)

ax[1][0].hist(stats_rnn['bleu_4'],bins=50)
ax[1][1].hist(stats_lstm['bleu_4'],bins=50)

ax[0][0].set_title('RNN')
ax[0][1].set_title('LSTM')

ax[0][0].set_ylabel('BLEU 1-GRAM')
ax[1][0].set_ylabel('BLEU Cumm. 4 GRAM')
fig.tight_layout(pad=1)
```



```
avg_bleu={}
avg_bleu['RNN']=stats_rnn[['bleu_1','bleu_4']].describe().iloc[1:3].to_dict()
avg_bleu['LSTM']=stats_lstm[['bleu_1','bleu_4']].describe().iloc[1:3].to_dict()
pd.DataFrame.from_dict({(i,j): avg_bleu[i][j]
                            for i in avg_bleu.keys()
                            for j in avg_bleu[i].keys()},
                       orient='index')
```

		mean	std
RNN	**bleu_1**	0.474831	0.162705
	bleu_4	0.090683	0.110181
LSTM	**bleu_1**	0.468501	0.161621
	bleu_4	0.088486	0.108984

The graph above represent the BLEU score of the prediction. Here we calculate two BLEU score:

1. BLEU-1 which is focusing on 1-Ngram. It means that if the more word in prediction match the corpus of references, it will produce the high score, since it only compare whether the word in prediction are in the corpus.
2. BLEU-4 which focus on 4 Ngram. Not only check wheter the word is on the corpus of reference, it also check whether the sequence of it match.

The left picture is for the RNN an the right one is the LSTM; while the top picture is the BLEU 1 and the bottom is the BLEU 4

Finding:

1. in BLEU-1, the distribution is quite similar with the RNN outperform LSTM with sligh differences. Although the standard error/variance of RNN is quite wide compared with LSTM, the differences is not noticable.
2. Similar distribution also found in BLEU-4 with RNN outperform the LSTM.

Here, we can say that concerning the BLEU score, the RNN is perform better than LSTM with slight differences.

5.2 Examplaire high and low score BLEU score samples (5 marks)

Find one sample with high BLEU score and one with a low score, and display the model's predicted sentences, the BLEU scores, and the 5 reference captions.

RNN

Bleu 4

High Score

```
nshape=stats_rnn.shape[0]
temp=stats_rnn.sort_values('bleu_4',ascending=False).iloc[:3]
temp['picture']=temp['file_name']
temp['ref']=temp.ref.apply(lambda y: ("".join([" ".join(i)+'<br>' for i in y[:5]])))
temp['preds']=temp.preds.apply(lambda y: " ".join(y).replace("<","&lt").replace(">",'&gt'))
HTML(temp[['picture','preds','ref','bleu_4','unk']].to_html(formatters={'picture': image_formatter}, escape=False))
```

	picture	preds	ref	bleu_4	unk
99		a red double decker bus driving down a street	a red double decker bus on street next to building a bus that is driving in the street a ride double decker bus stands out against a black and white background a double decker bus with few passengers turning at a corner a red double decker bus driving down a city street	0.889140	0
643		a group of people standing around a table	a group of people standing around a kitchen next to appliances several people busy in a kitchen cooking food men and women in aprons working in a kitchen a group of people in a kitchen preparing food a group of men and women around an island in a commercial kitchen	0.840896	0
38		a group of motorcycles parked on a sidewalk street	a man riding a bike past a rows of parked motor cycles a long row of parked motorcycles and a bicyclist riding past them a group of motorcycles are lined up against the sidewalk a row of motorcycles parked on a city street a bunch of motorcycles are in a parking lot	0.719409	0

Lowest Score

```
nshape=stats_rnn.shape[0]
temp=stats_rnn.sort_values('bleu_4',ascending=True).iloc[:3]
temp['picture']=temp['file_name']
temp['ref']=temp.ref.apply(lambda y: ("".join([" ".join(i)+'<br>' for i in y[:5]])))
temp['preds']=temp.preds.apply(lambda y: " ".join(y).replace("<","&lt").replace(">",'&gt'))
HTML(temp[['picture','preds','ref','bleu_4','unk']].to_html(formatters={'picture': image_formatter}, escape=False))
```

	picture	preds	ref	bleu_4	unk
458		a close up of a stop sign in front of a building and blue fire hydrant	a damaged leather suit case sitting on a dirty sidewalk a box sitting on a dirty street surrounded by garbage a trunk on a sidewalk showing warping from water damage a brown suit case thrown on a cluttered sidewalk writing painted on to an old suit case by a dump	0.014628	0
756		a man in a black shirt and black shirt and a woman on a <unk>	a group of people holding some hot dogs in his hand a group of people are holding corn dogs in containers people hold six corn dogs with various mustard designs corn dog sticks in tray held together by hands a group of people are showing their corndogs with designs from mustard on them	0.017396	1
358		a young man in a black shirt and black shirt and a woman on a <unk>	a person standing in front of an elephant that is laying down a mam holding the tail of an elephant a man with an instrument holds an elephant s trunk a wildlife manager examines an elephant laying on it s side a man takes a picture of an elephant s trunk	0.018394	1

Bleu 1

High score

```
nshape=stats_rnn.shape[0]
temp=stats_rnn.sort_values('bleu_1',ascending=False).iloc[:3]
temp['picture']=temp['file_name']
temp['ref']=temp.ref.apply(lambda y: ("".join([" ".join(i)+'<br>' for i in y[:5]])))
temp['preds']=temp.preds.apply(lambda y: " ".join(y).replace("<","&lt").replace(">",'&gt'))
HTML(temp[['picture','preds','ref','bleu_1']].to_html(formatters={'picture': image_formatter}, escape=False))
```

	picture	preds	ref	bleu_1
27		a large bus is parked on the side of a road	a large bus that is parked on the street next to the sidewalk a bus that is next to a biulding a city bus sitting out side of down town stop a tour bus with a wi fi notice parked on the side of the road	1.0
39		a double decker bus is driving down the street	a double decker bus driving past a bunch of tall buildings a large long bus on a city street a double decker bus drives down the city street a double decker bus is going through an old neighborhood a orange and cream colored double layered bus is on a lane near white buildings	1.0
99		a red double decker bus driving down a street	a red double decker bus on street next to building a bus that is driving in the street a ride double decker bus stands out against a black and white background a double decker bus with few passengers turning at a corner a red double decker bus driving down a city street	1.0

```
nshape=stats_rnn.shape[0]
temp=stats_rnn.sort_values('bleu_1',ascending=True).iloc[:3]
temp['picture']=temp['file_name']
temp['ref']=temp.ref.apply(lambda y: ("".join([" ".join(i)+'<br>' for i in y[:5]])))
temp['preds']=temp.preds.apply(lambda y: " ".join(y).replace("<","&lt").replace(">",'&gt'))
HTML(temp[['picture','preds','ref','bleu_1']].to_html(formatters={'picture': image_formatter}, escape=False))
```

	picture	preds	ref	bleu_1
459		the birthday cake for <unk> <unk> <unk> <unk> <unk> <unk>	a suit case filled with a magazine and a pair of shoes an open suitcase on the ground containing a pair of shoes and a magazine shoes and a magazine lie scattered inside an open suitcase an open suit case with a magazine and one show a suitcase containing only a pair of shoes and a magazine	0.100000
528		a person <unk> a picture of a <unk> <unk>	little girl observing three octopus kites being flown a girl watching kites shaped like octopuses a little girl watching colorful kites in the sky a young child watches octopus shaped kites fly the child is flying a kite on the beach	0.111111
398		a person is riding a bike down a river	two giraffes looking up at an awning for food there are two adult giraffe standing together in the den two giraffes in an enclosure reaching for the gutter on the roof two giraffes standing next to a building in an enclosure two giraffes stretching up toward the roof of a building	0.111111

LSTM

Bleu 4

Highest Score

[ ] ↳ 7 cells hidden

Finding

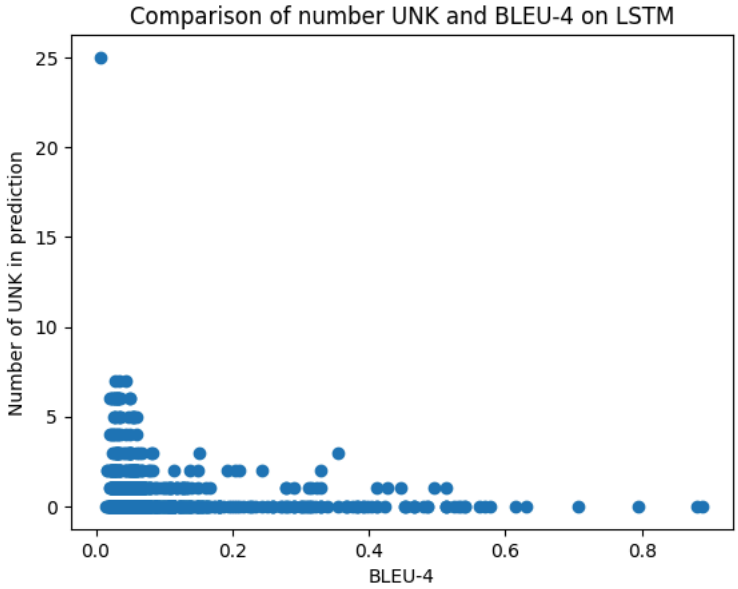
Refer to the example of prediction above, the RNN can give better prediction compared with the LSTM.

Sorting by the score on descending order, RNN can give a good prediction of "Bus parked on the road". However, the when checking the low score example, we note that there is much of <unk> word. Here we try to compare the BLEU score and the number of UNK. We will focus on bleu_4 instea of bleu_1

```
plt.scatter(stats_lstm.bleu_4,stats_lstm.unk)
plt.xlabel('BLEU-4')
```

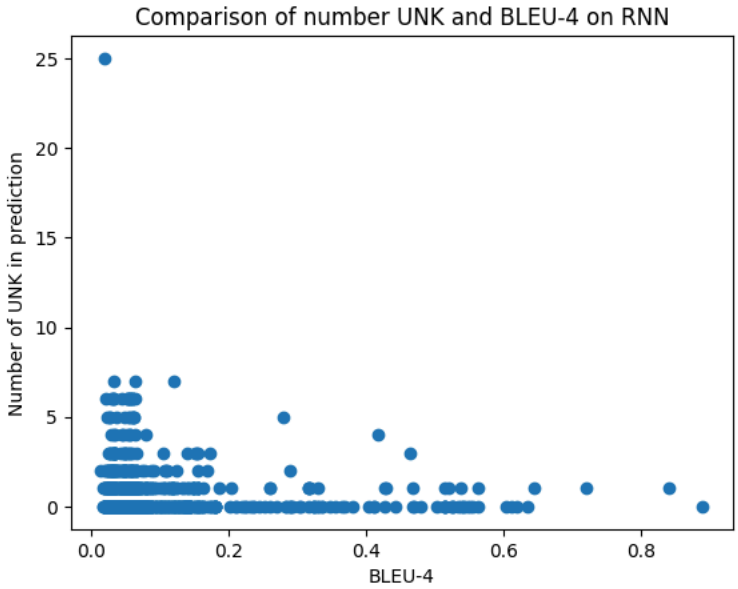
```
plt.ylabel('Number of UNK in prediction')
plt.title('Comparison of number UNK and BLEU-4 on LSTM')
```

```
Text(0.5, 1.0, 'Comparison of number UNK and BLEU-4 on LSTM')
```




```
plt.scatter(stats_rnn.bleu_4,stats_lstm.unk)
plt.xlabel('BLEU-4')
plt.ylabel('Number of UNK in prediction')
plt.title('Comparison of number UNK and BLEU-4 on RNN')
```

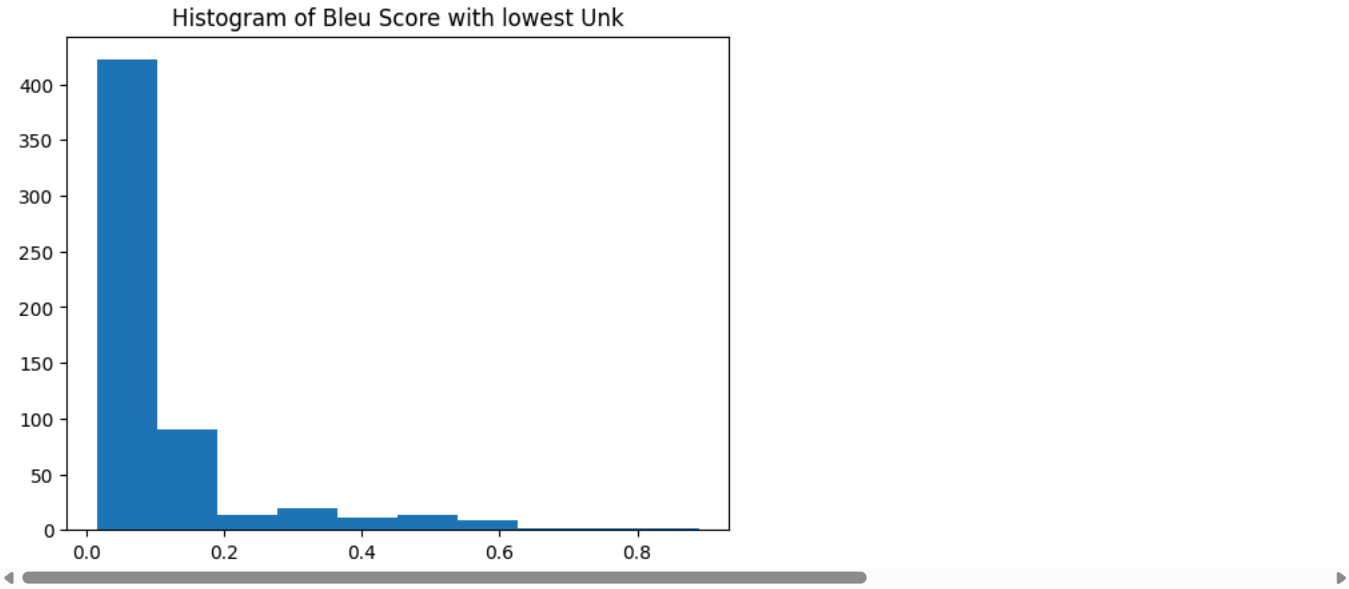
```
Text(0.5, 1.0, 'Comparison of number UNK and BLEU-4 on RNN')
```



Here we note that the for the higher unk number the bleu-4 distribution score is shifted near 0 and the variation is shrinking. Let try to compare the Bleu-4 score for number of unk is low (0) an high (5).

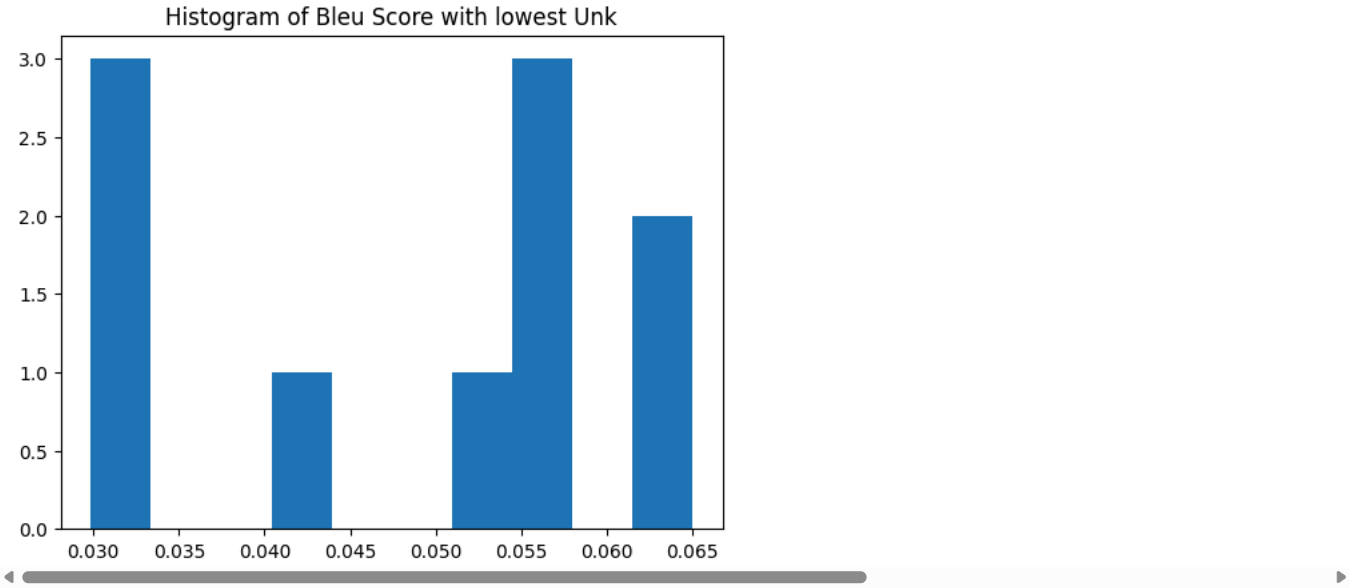
```
plt.hist(stats_rnn.query('unk==0').bleu_4)
plt.title('Histogram of Bleu Score with lowest Unk')
```

```
Text(0.5, 1.0, 'Histogram of Bleu Score with lowest Unk')
```




```
plt.hist(stats_rnn.query('unk==5').bleu_4)
plt.title('Histogram of Bleu Score with lowest Unk')
```

```
Text(0.5, 1.0, 'Histogram of Bleu Score with lowest Unk')
```



Here, we found that the range of BLEU Score shrink from [0,0.8] to [0.03,0.06]. Therefore we can say that the more number of <unk> which predicted, the lower is bleu score.

6 Caption evaluation using cosine similarity [12 marks]

> 6.1 Cosine similarity (6 marks)

> 6.2 Cosine similarity examples (6 marks)

6.1 Cosine similarity (6 marks)

The cosine similarity measures the cosine of the angle between two vectors in n-dimensional space. The smaller the angle, the greater the similarity.

To use the cosine similarity to measure the similarity between the generated caption and the reference captions:

- Find the embedding vector of each word in the caption
- Compute the average vector for each caption
- Compute the cosine similarity score between the average vector of the generated caption and average vector of each reference caption
- Compute the average of these scores

Calculate the cosine similarity using the model's predictions over the whole test set.

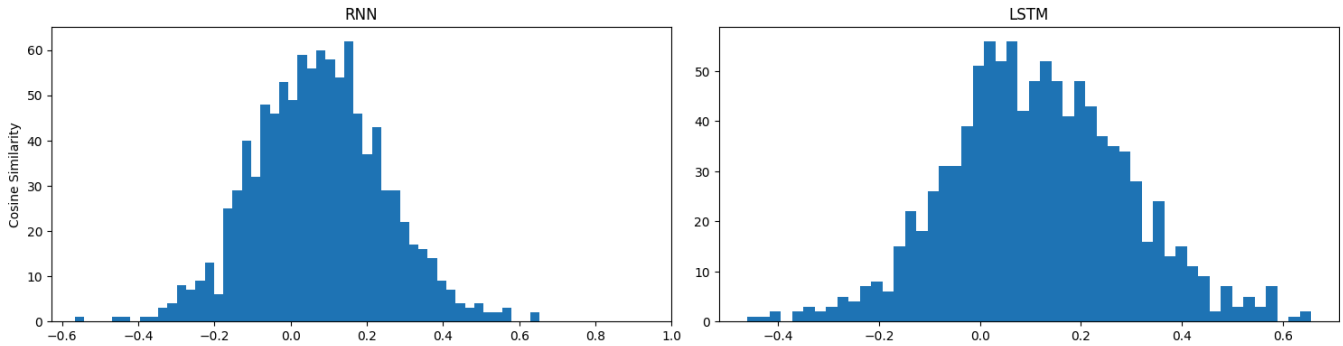
Display a histogram of the distribution of scores over the test set.

```
def sentence_to_vocab(sentence,vocab,min_seq):
  y_=[0]*min_seq
  sent_vocab=[vocab.__call__(i) for i in sentence.split(' ')]
  if len(sent_vocab)>=min_seq:
    sent_vocab[(min_seq-1)]=2
  else:
    sent_vocab.append(2)
  index_end=np.where(np.array(sent_vocab)==2)[0][0]
  for i in range(index_end+1):
    y_[i]=sent_vocab[i]
  return y_


def find_embed_vector(sentence,vocab,net,min_seq):

  sentence=sentence_to_vocab(sentence,vocab,min_seq)
  y_=net.embed(torch.tensor(sentence))
  return y_


def cosine_calc(ref,hyp,vocab,net):
  sim_=[]
  #net=nn.Embedding(vocab.__len__(),256)
  net.cpu()
  cos_=torch.nn.CosineSimilarity(dim=0)
  cos_.cpu()
  for i in ref:
    min_seq=max(len(i.split()),len(hyp.split()))
    ref_=find_embed_vector(i,vocab,net,min_seq)
    ref_=torch.mean(ref_,dim=1)
    hyp_=find_embed_vector(hyp,vocab,net,min_seq)
    hyp_=torch.mean(hyp_,dim=1)
    t_sim=cos_(torch.tensor(ref_.tolist()),
               torch.tensor(hyp_.tolist()))
    y_=t_sim.item()
    sim_.append(y_)
  return np.mean(sim_)


stats_rnn['cos_sim']=test_predict_.apply(lambda y: cosine_calc(y.clean_sample,y.rnn_pred,vocab,decoderRnn),axis=1)
stats_lstm['cos_sim']=test_predict_.apply(lambda y: cosine_calc(y.clean_sample,y.lstm_pred,vocab,decoderLstm),axis=1)
```

The code above represent the cosine similarity function. The code run as follow:

1. Convert the sentence into list of number, representing the word in vocab.However since the length of each caption is different with the length of prediction, we try to make use the max length of caption vs prediction every comparison. For instance: a. if one reference have length [9,7] and the prediction have length [8]. for the first comparison we use the length equals to 9 and for the second comparison we will use length equals to 8
2. Make the embedding vector; here we use the embedding layer of decoder we train before in evaluation mode.
3. Calculate the cosine

```
fig,ax=plt.subplots(1,2,figsize=(15,4))
ax[0].hist(stats_rnn['cos_sim'],bins=50)
ax[1].hist(stats_lstm['cos_sim'],bins=50)

ax[0].set_title('RNN')
ax[1].set_title('LSTM')

ax[0].set_ylabel('Cosine Similarity')
ax[0].set_xlim(right=1)
fig.tight_layout(pad=1)
```




```
avg_sim={}
avg_sim['RNN']=stats_rnn[['cos_sim']].describe().iloc[1:3].to_dict()
avg_sim['LSTM']=stats_lstm[['cos_sim']].describe().iloc[1:3].to_dict()
pd.DataFrame.from_dict({(i,j): avg_sim[i][j]
                        for i in avg_sim.keys()
                        for j in avg_sim[i].keys()},
                       orient='index')
```

		mean	std
RNN	**cos_sim**	0.074882	0.169784
LSTM	**cos_sim**	0.113818	0.176730

The graph above illustrate that the distribution of cossine similarity from the RNN and LSTM. From distribution, it looks that the LSTM outperform the RNN with huge differences. The average of cosine similarity in LSTM is higher than the RNN yet its standar error is quite similar. Hence it makes the LSTM give better prediction based on cosine similarity.

6.2 Cosine similarity examples (6 marks)

Find one sample with high cosine similarity score and one with a low score, and display the model's predicted sentences, the cosine similarity scores, and the 5 reference captions.

RNN

[ ] ↳ 3 cells hidden

LSTM

[ ] ↳ 2 cells hidden

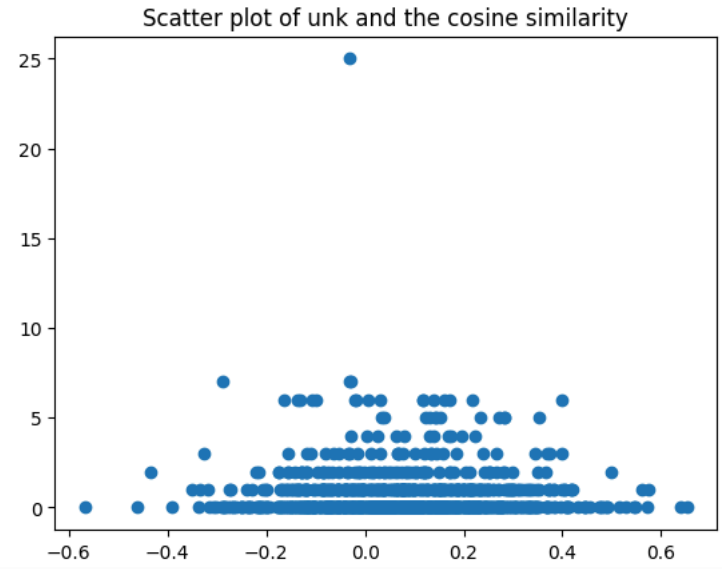
Finding

It turn out that LSTM give better prediction with remaks to the presences of "unk" as shown in the first row: "a <unk> <unk> <unk> <unk> on a <unk> day".

It looks that in cosine similarity, since we change the caption into embeddingvector, there is possibility the caption reference also have "unk" in it. Lets try put scatter plot of the number of unk and the cosine similairty. Here we focus on the result of LSTM

```
plt.scatter(stats_rnn.cos_sim,stats_lstm.unk)
plt.title('Scatter plot of unk and the cosine similarity')
```

```
Text(0.5, 1.0, 'Scatter plot of unk and the cosine similarity')
```



It looks the mean is not shifting yet the variation is shrinken. Here is some example of predictions sorting on number of unk

```
temp=stats_lstm.sort_values('unk',ascending=False).iloc[:3]
temp['picture']=temp['file_name']
temp['ref']=temp.ref.apply(lambda y: ("".join([" ".join(i)+'<br>' for i in y[:5]])))
temp['preds']=temp.preds.apply(lambda y: " ".join(y).replace("<","&lt").replace(">",'&gt'))
HTML(temp[['picture','preds','ref','cos_sim','unk']].to_html(formatters={'picture': image_formatter}, escape=False))
```

	picture	preds	ref	cos_sim	unk
680		a <unk> <unk> <unk> <unk> into a <unk> <unk> of <unk> <unk> <unk> <unk> <unk> <unk> <unk> <unk> <unk> <unk> <unk> <unk> <unk> <unk> <unk> <unk> <unk> <unk> <unk>	three bananas with unique ripeness are aligned in a round fashion a white basket filled with ripe and unripe bananas three banana in different stages of ripening in circular arrangement in in light brown tone cloth woven basket there is a bowl with three bananas in them a basket with three bananas in various stages of ripeness	0.135864	25
768		a <unk> <unk> <unk> a <unk> <unk> of a <unk> <unk>	an assortment of donuts with various coatings and toppings many donuts are sitting on a wooden counter there are many different donuts sitting on top of a table a selection of doughnuts on a wooden counter a variety of donuts are sitting on top of a wooden table	0.009514	7
			a man playing a first person shooter video game a person holding a white xbox video game controller with the game on the television in		

```
## we try to check the first picture "the banana"
temp=stats_rnn.sort_values('unk',ascending=False).iloc[0]
sim_=[]
  #net=nn.Embedding(vocab.__len__(),256)

cos_=torch.nn.CosineSimilarity(dim=0)
iter_=0
for i in temp.ref:
  print('THE ',iter_+1," COMPARISON")
  print('Prediction Embedding')
  min_seq=max(len(i),len(temp.preds))
  print('Sentence to vocab:',sentence_to_vocab(" ".join(temp.preds),vocab,min_seq))
  hyp=find_embed_vector(" ".join(temp.preds),vocab,decoderRnn,min_seq)
  print(hyp)
  hyp_=torch.mean(hyp,dim=1)

  print('References: ')
  print('Sentence to vocab:',sentence_to_vocab(" ".join(i),vocab,min_seq))
```

```
  ref_=find_embed_vector(" ".join(i),vocab,decoderLstm,min_seq)
  print(ref_)

  ref_=torch.mean(ref_,dim=1)
  t_sim=cos_(ref_,hyp_)
  t_sim.cpu()
  y_=t_sim.item()
  sim_.append(y_)
  iter_+=1
print(sim_)
```

```
THE  1  COMPARISON
Prediction Embedding
Sentence to vocab: [5, 1, 1, 1, 1, 1, 5, 1, 2]
tensor([[ 0.1441,  1.3104, -0.1688,  ...,  0.5444,  0.6132, -0.0159],
        [-1.7467,  0.2247, -0.5229,  ...,  0.2886, -0.7368, -1.0435],
        [-1.7467,  0.2247, -0.5229,  ...,  0.2886, -0.7368, -1.0435],
        ...,
        [ 0.1441,  1.3104, -0.1688,  ...,  0.5444,  0.6132, -0.0159],
        [-1.7467,  0.2247, -0.5229,  ...,  0.2886, -0.7368, -1.0435],
        [-0.6616, -0.3269,  0.6326,  ..., -0.2138,  0.6588,  0.0871]],
       grad_fn=<EmbeddingBackward0>)
References:
Sentence to vocab: [42, 391, 311, 5, 606, 12, 202, 593, 2]
tensor([[-3.2376e-01,  1.0670e+00,  7.4992e-01,  ..., -6.9231e-01,
         -1.0623e+00,  1.4693e+00],
        [ 6.6067e-02,  1.9432e+00, -1.0642e+00,  ...,  7.0826e-02,
          1.5501e+00, -6.2980e-01],
        [ 1.6130e+00,  1.4493e-01, -4.2676e-01,  ...,  7.2271e-01,
         -8.7416e-01, -1.6726e+00],
        ...,
        [-1.9173e-01, -5.5212e-01, -1.8949e+00,  ...,  9.0322e-01,
          6.5738e-01,  1.5750e+00],
        [-1.2876e+00, -3.3959e-01, -2.2670e-01,  ...,  6.7375e-01,
         -5.7928e-01,  7.2533e-01],
        [-6.6101e-01,  8.6570e-02, -2.2306e-03,  ...,  9.9386e-04,
         -6.4271e-01,  8.3186e-01]], grad_fn=<EmbeddingBackward0>)
THE  2  COMPARISON
Prediction Embedding
Sentence to vocab: [5, 1, 1, 1, 1, 1, 5, 1, 2]
tensor([[ 0.1441,  1.3104, -0.1688,  ...,  0.5444,  0.6132, -0.0159],
        [-1.7467,  0.2247, -0.5229,  ...,  0.2886, -0.7368, -1.0435],
        [-1.7467,  0.2247, -0.5229,  ...,  0.2886, -0.7368, -1.0435],
        ...,
        [ 0.1441,  1.3104, -0.1688,  ...,  0.5444,  0.6132, -0.0159],
        [-1.7467,  0.2247, -0.5229,  ...,  0.2886, -0.7368, -1.0435],
        [-0.6616, -0.3269,  0.6326,  ..., -0.2138,  0.6588,  0.0871]],
       grad_fn=<EmbeddingBackward0>)
References:
Sentence to vocab: [42, 97, 391, 4, 379, 5, 621, 606, 2]
tensor([[-3.2376e-01,  1.0670e+00,  7.4992e-01,  ..., -6.9231e-01,
         -1.0623e+00,  1.4693e+00],
        [ 4.3472e-02, -9.9577e-01, -8.3415e-01,  ...,  2.9294e-01,
          1.1680e-01,  3.2771e-01],
        [ 6.6067e-02,  1.9432e+00, -1.0642e+00,  ...,  7.0826e-02,
          1.5501e+00, -6.2980e-01],
        ...,
        [-4.2578e-01, -3.8750e-01,  4.8115e-02,  ...,  5.2403e-02,
         -5.5733e-01, -6.1901e-01],
        [-2.6731e-01,  2.2056e-01,  6.1536e-02,  ...,  1.7092e-01,
         -4.2804e-02, -7.1786e-01],
        [-6.6101e-01,  8.6570e-02, -2.2306e-03,  ...,  9.9386e-04,
         -6.4271e-01,  8.3186e-01]], grad_fn=<EmbeddingBackward0>)
THE  3  COMPARISON
Prediction Embedding
Sentence to vocab: [5, 1, 1, 1, 1, 1, 5, 1, 1, 2, 0]
tensor([[ 0.1441,  1.3104, -0.1688,  ...,  0.5444,  0.6132, -0.0159],
        [-1.7467,  0.2247, -0.5229,  ...,  0.2886, -0.7368, -1.0435],
        [-1.7467,  0.2247, -0.5229,  ...,  0.2886, -0.7368, -1.0435],
```

The code above represent the embedding of prediction of the first picture and the reference. Here we found that:

- the first caption is quite similar with our prediction
- the fourth caption is so differnt with our prediction

Besides that, the padding value also affecting the calculation. Here we found that it is intends to "similar" when the caption and the prediction have similar length. Hence the cosine similarity also affected by the number of pad added on the sentence.

7 Comparing BLEU and Cosine similarity [16 marks]

> 7.1 Test set distribution of scores (6 marks)

> 7.2 Analysis of individual examples (10 marks)

7.1 Test set distribution of scores (6 marks)

Compare the model's performance on the test set evaluated using BLEU and cosine similarity and discuss some weaknesses and strengths of each method (explain in words, in a text box below).

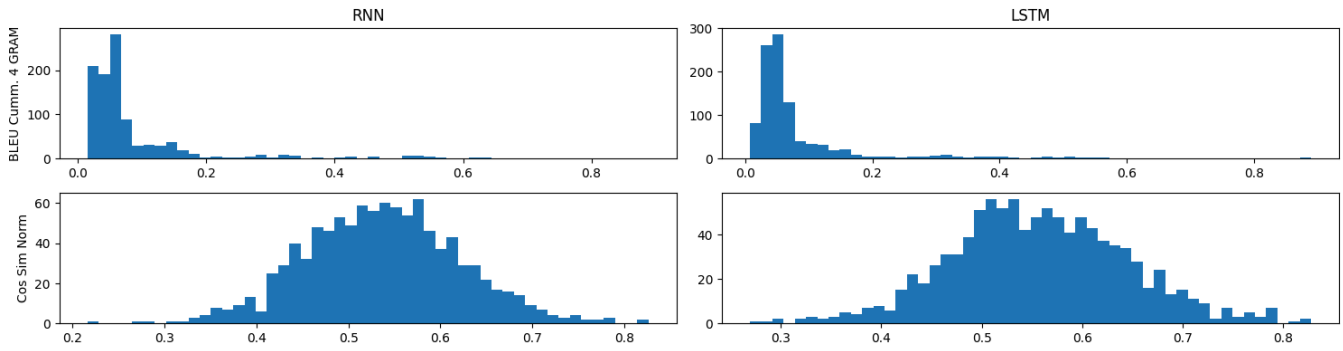
Please note, to compare the average test scores, you need to rescale the Cosine similarity scores [-1 to 1] to match the range of BLEU method [0.0 - 1.0].

```
stats_rnn['cos_sim_norm']=stats_rnn.cos_sim.apply(lambda y: (y+1)/2)
stats_lstm['cos_sim_norm']=stats_lstm.cos_sim.apply(lambda y: (y+1)/2)
```

```
avg_all={}
avg_all['RNN']=stats_rnn[['bleu_1','bleu_4','cos_sim','cos_sim_norm']].describe().iloc[1:].to_dict()
avg_all['LSTM']=stats_lstm[['bleu_1','bleu_4','cos_sim','cos_sim_norm']].describe().iloc[1:].to_dict()
pd.DataFrame.from_dict({(i,j): avg_all[i][j]
                        for i in avg_all.keys()
                        for j in avg_all[i].keys()},
                       orient='index')
```

		mean	std	min	25%	50%	75%	max
RNN	**bleu_1**	0.474831	0.162705	0.100000	0.357143	0.444854	0.582199	1.000000
	bleu_4	0.090683	0.110181	0.014628	0.033913	0.056376	0.080991	0.889140
	cos_sim	0.074882	0.169784	-0.567138	-0.041664	0.074053	0.184342	0.652463
	cos_sim_norm	0.537441	0.084892	0.216431	0.479168	0.537027	0.592171	0.826232
LSTM	**bleu_1**	0.468501	0.161621	0.068966	0.354412	0.444444	0.555556	1.000000
	bleu_4	0.088486	0.108984	0.007696	0.033913	0.053965	0.079369	0.889140
	cos_sim	0.113818	0.176730	-0.461810	-0.003980	0.107444	0.230171	0.656069
	cos_sim_norm	0.556909	0.088365	0.269095	0.498010	0.553722	0.615086	0.828034

```
fig,ax=plt.subplots(2,2,figsize=(15,4))
ax[0][0].hist(stats_rnn['bleu_4'],bins=50)
ax[0][1].hist(stats_lstm['bleu_4'],bins=50)

ax[1][0].hist(stats_rnn['cos_sim_norm'],bins=50)
ax[1][1].hist(stats_lstm['cos_sim_norm'],bins=50)

ax[0][0].set_title('RNN')
ax[0][1].set_title('LSTM')

ax[0][0].set_ylabel('BLEU Cumm. 4 GRAM')
ax[1][0].set_ylabel('Cos Sim Norm')
fig.tight_layout(pad=1)
```



The graph above represent the comparison of RNN and LSTM in standardised cosines and BLEU-4. It turns out that the distribution in RNN is quite shifted to left compared ith LSTM.

From the table above the graph, we can also say that:

- RNN perform best based on BLEU

- LSTM perform best based on Cosine Similarity

However, since the difference in BLEU indicator is quite low, we can say that LSTM is perform better than RNN

7.2 Analysis of individual examples (10 marks)

Find and display one example where both methods give similar scores and another example where they do not and discuss. Include both scores, predicted captions, and reference captions.

```
stats_rnn['difference']=abs(stats_rnn['bleu_4']-stats_rnn['cos_sim_norm'])
stats_lstm['difference']=abs(stats_lstm['bleu_4']-stats_lstm['cos_sim_norm'])
```

The most similar (denoted with lower absolute difference)

```
nshape=stats_lstm.shape[0]
temp=stats_rnn.sort_values('difference',ascending=True).iloc[:3]
temp['picture']=temp['file_name']
temp['ref']=temp.ref.apply(lambda y: ("".join([" ".join(i)+'<br>' for i in y[:5]])))
temp['preds']=temp.preds.apply(lambda y: " ".join(y).replace("<","&lt").replace(">",'&gt'))
HTML(temp[['picture','preds','ref','cos_sim_norm','bleu_4','bleu_1']].to_html(formatters={'picture': image_formatter},
```

	picture	preds	ref	cos_sim_norm	bleu_4	bleu_1
117		a <unk> <unk> on the side of a road	graffiti is painted on the side of a train a train car with graphittie on it is stopped in the station a view of a some graffiti on a train passing by graffiti on a train car that is parked at a station a train sitting in a train station covered in graffiti	0.465874	0.467138	0.666667
0		a man riding a skateboard on a beach	the skateboarder is putting on a show using the picnic table as his stage a skateboarder pulling tricks on top of a picnic table a man riding on a skateboard on top of a table a skate boarder doing a trick on a picnic table a person is riding a skateboard on a picnic table with a crowd watching	0.513906	0.520815	0.681451
309		a motorcycle parked on a city street with a <unk>	motorcycle parked along the street next to a building a motorcycle parked next to other motorcycles on city street a motorcycle is parker on a city sidewalk a black motorcycle is on display with more on	0.514682	0.525382	0.800000

```
nshape=stats_lstm.shape[0]
temp=stats_lstm.sort_values('difference',ascending=True).iloc[:3]
temp['picture']=temp['file_name']
temp['ref']=temp.ref.apply(lambda y: ("".join([" ".join(i)+'<br>' for i in y[:5]])))
temp['preds']=temp.preds.apply(lambda y: " ".join(y).replace("<","&lt").replace(">",'&gt'))
HTML(temp[['picture','preds','ref','cos_sim_norm','bleu_4','bleu_1']].to_html(formatters={'picture': image_formatter},
```

	picture	preds	ref	cos_sim_norm	bleu_4	bleu_1
137		a man is on a boat in the water	a man on a boat sailing on the ocean a man in a boat on foggy water the man in the cowboy hat is in a boat in the water a man sitting in a boat on very still water a person in a boat with a hat water and a hill	0.626291	0.614788	1.000000
417		a group of people standing around a table	a young person flying a kite at the beach a boy on a beach flying a kite with other people around a child flies a kite on a beach	0.534349	0.562341	0.875000